Submitted by: Vismaya T M

MindMatrixInternship-Android App Development Using Gen AI

UI Testing and Art Navigation

Task 1 — Automated Testing Deep Dive & Mapping

Introduction

This task explores automated testing strategies for my implemented Art Space App built using Jetpack Compose. The app displays artwork images along with title, artist, and year information. Users can navigate between artworks using "Previous" and "Next" buttons. Since Compose follows a state-driven model, testing is necessary to ensure that UI updates correctly when state changes. This document defines testing concepts, selects a feature to test, and maps the UI behavior to appropriate test types.

1. Research & Define

1. Unit Test
   A unit test verifies small pieces of business logic independently from the UI or Android framework.

Example from my app:
Testing the function that calculates the next artwork index when the user presses the "Next" button.

Example logic:

```
fun getNextIndex(currentIndex: Int, total: Int): Int {
    return (currentIndex + 1) % total
}
```

This logic can be tested without launching the UI.

2. Instrumented UI Test
   An instrumented UI test runs on a device or emulator and verifies actual UI behavior and user interaction.

Example from my app:
Verifying that pressing the "Next" button changes:

- The displayed image
- The artwork title

- The artist and year text

3. ComposeTestRule
   ComposeTestRule is a testing rule that launches Compose UI during tests and allows interaction with UI elements.

Example:

@get:Rule

val composeTestRule = createComposeRule()

It enables:

- onNodeWithText("Next").performClick()
- assertExists()
- assertIsDisplayed()

4. Semantics Tree
   The semantics tree represents UI elements in a structure accessible to tests and accessibility services.

In my Art Space app:

- “Next” and “Previous” buttons are accessible by text.
- The artwork title is accessible as a Text node.
- The image can be identified using contentDescription.

2. Selected UI Feature to Test

Feature: Next Artwork Button

Description of behavior:

When the user taps “Next”:

1. currentArtworkIndex increases by 1.
2. The displayed image changes.
3. The artwork title updates (for example, from “Sailing Under the Bridge” to the next artwork).
4. The artist and year text update.
5. If the current artwork is the last one, the index wraps to the first artwork.

UI elements involved:

- Image (Artwork display)
- Text (Artwork title)

- Text (Artist and year)
- Button ("Previous")
- Button ("Next")

3. Test Strategy Document

Test Type	What to Test	Setup Needed	Assertion / Verification
Unit Test	getNextIndex logic	None	Index increments correctly
Unit Test	Wrap-around logic	currentIndex = last	Returns 0
UI Test	Tap Next button	Launch ArtSpaceScreen	Title text updates
UI Test	Tap Previous button	Index > 0	Previous artwork shown
UI Test	Initial UI state	App launched	First artwork displayed

Justification:

1. Index calculation is pure logic and independent of UI → Unit Test.
2. Button interaction and UI update require Compose rendering → Instrumented UI Test.
3. Verifying image and text updates requires semantics tree → UI Test.

4. Reflection:

Writing tests early improves code quality because it clearly defines expected behavior before implementation. If state changes but UI is not verified, recomposition issues may go unnoticed. For example, the index might update correctly but the image might not refresh due to incorrect state management. One anticipated challenge in Compose UI testing is uniquely identifying UI nodes when multiple Text components exist. Proper content descriptions and structured state separation help reduce this issue.

Task 2 — Art Space App: Design, States, & Test Plan

1. Feature & UI Design

My Art Space App includes:

1. Gallery View:
    - Displays artwork image.
    - Shows title, artist, and year.
    - Includes “Previous” and “Next” navigation buttons.
2. The main feature:
    - Users browse artworks using navigation buttons.
    - The displayed content updates dynamically based on currentArtworkIndex.

Sketch Description (Based on Implemented UI):

- Large centered artwork image.
- Title below image.
- Artist and year below title.
- Two buttons at bottom: Previous and Next.

2. State Variables & Logic

Key state variables:

1. currentArtworkIndex: Int
   Initial value: 0
   Transitions:
    - Tap Next → currentArtworkIndex++
    - Tap Previous → currentArtworkIndex--
    - If index == last → wrap to 0
    - If index == 0 and Previous tapped → wrap to last
2. artworks: List
   Contains image resource, title, artist, and year.
3. currentArtwork (Derived State)
   Derived from artworks[currentArtworkIndex].

3. Component & Testable Units

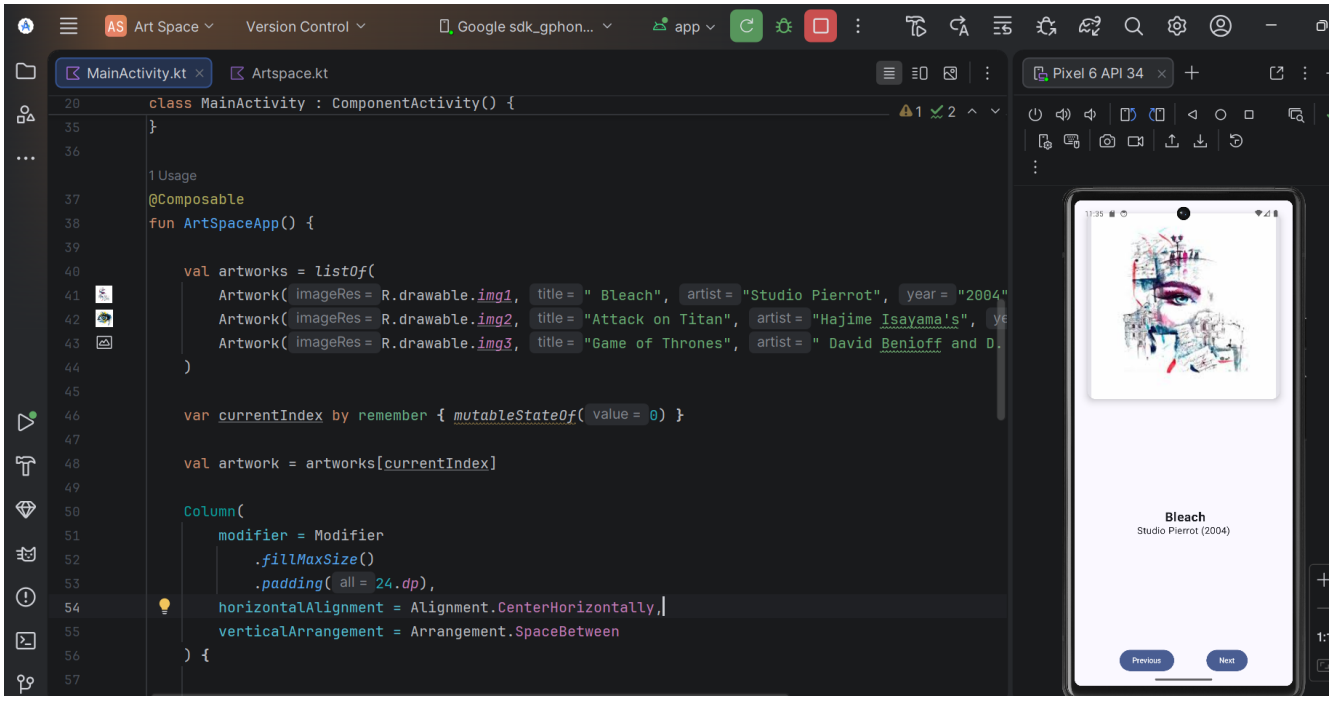
1. Next Button Advances Artwork
   - Component: Button("Next")
   - Trigger: performClick()
   - Expected Result: New image and title displayed
2. Previous Button Moves Back
   - Component: Button("Previous")
   - Trigger: performClick()
   - Expected Result: Previous artwork displayed
3. Artwork Display Synchronization
   - Component: Image + Text
   - Trigger: State change
   - Expected Result: Image, title, and metadata match the same artwork

4. Test Plan Table

Test ID	Unit Under Test	Test Type	Preconditions	Trigger	Expected Outcome
T1	Initial Display	UI	App launched	None	First artwork shown
T2	Next Navigation	UI	Index = 0	Click Next	Second artwork displayed
T3	Previous Navigation	UI	Index = 1	Click Previous	First artwork displayed
T4	Wrap Forward	Unit	Index = last	Call getNextIndex	Returns 0
T5	Wrap Backward	Unit	Index = 0	Call getPreviousIndex	Returns last index

Edge Case Covered:

- Navigation wrapping at boundaries.
- No crash when navigating at first or last artwork.



5. Reflection

Planning tests alongside design improved the reliability of the app because state transitions were clearly defined before implementation. Business logic tests were easier to write compared to UI tests, which required node identification and ComposeTestRule setup. To reduce UI test flakiness, logic was separated from composables and state was managed centrally.

Task 3 — Implementation Reflection & Test Coverage Audit

1. Implementation Summary

Implemented Features:

1. Next button navigation.
2. Previous button navigation.
3. Dynamic artwork display update.

Tests Written:

1. Unit test for next index wrap-around.
2. Unit test for previous index wrap-around.
3. UI test verifying Next button updates artwork.
4. UI test verifying Previous button updates artwork.

2. Test Coverage Audit

Component	Has Test?	Type of Test	Coverage Status	Priority	Comments
Next Button	Yes	Unit + UI	Good	High	Core feature covered
Previous Button	Yes	Unit + UI	Good	High	Wrap logic verified

Artwork Image Display	Partial	UI	Needs Improvement	Medium	Add contentDescription test
Metadata Text (Title/Year)	Yes	UI	Good	Medium	Verified title change
Orientation Handling	No	None	Needs Improvement	High	State resets on rotation

3. Bug / Edge-Case Discovery

Identified Issue:
On device rotation, currentArtworkIndex resets to 0 because remember was used instead of rememberSaveable.

Proposed Solution:
Replace remember with rememberSaveable for currentArtworkIndex.

Proposed New Test:
Simulate configuration change and verify that the same artwork remains visible.

4. Reflection

Writing automated tests changed my development mindset. I began designing logic in testable units instead of embedding logic directly inside composables. The main challenges were UI node identification and handling recomposition timing. In future implementations, I will prioritize better state preservation, use ViewModel for state management, and improve coverage for configuration changes.

Conclusion

Through designing, implementing, and testing my Art Space App, I developed a strong understanding of how state, UI behavior, and automated testing work together in Jetpack Compose. By separating logic from UI and mapping features to proper test types, I ensured reliable navigation and consistent state-driven updates. Automated testing strengthened the overall structure, maintainability, and confidence in the app's behavior.

Submitted by: Vismaya T M

MindMatrixInternship-Android App Development Using Gen AI